

プログラミングの基礎 1

はじめに

- この一連のノートは以下の内容をまとめます。
- Pythonの使い方
- 微分方程式と数値計算の基本

学習目標

- Pythonを実行できるようになる。
- 変数の代入を理解する。
- 代表的なデータ型を扱えるようになる。

プログラミング言語

● 機械語

- コンピュータが理解できるように書かれた言語で人間には理解しづらい（0と1で表現される）

● プログラミング言語

- コンピュータに指示を与えるために人間に理解しやすいように設計された言語
- C言語、Java, Python, などなど

● 人間が書いたプログラムを機械語に翻訳する必要がある。



プログラミング言語から 機械語への翻訳

- ◎ 翻訳の仕方は2通りある。
- ◎ コンパイル
 - ソースコードを一括して機械語に翻訳する。一括翻訳してから実行する。
 - コンパイル時に文法エラーなどを検出してくれる。
 - 代表例：C言語,Java
- ◎ インタプリタ
 - ソースコードを一行ずつ機械語に翻訳しながら実行する。
 - コードの途中で文法エラーがあっても、エラーがある手前までは実行できる。
 - 代表例：Python

Pythonの実行

◎ Pythonをターミナル上で実行してみます。

- 対話型での実行
 - ターミナル上でpython或いはpython3と入力
 - `print("Hello")`と入力してエンター
 - 対話型実行を抜け出すには`exit()`と入力してエンター
 - `print`関数は引数内の文字列を出力
 - 他にも計算結果も出力される

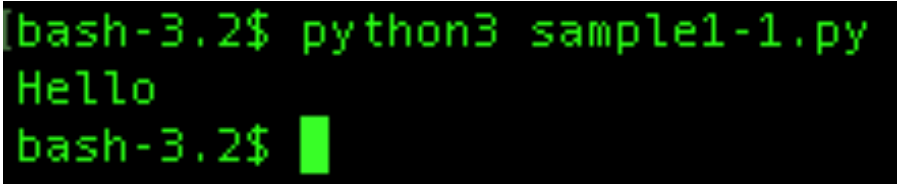
入力 →
出力 →

```
bash-3.2$ python3
Python 3.13.0 (main, Oct 7 2024, 05:02:14) [Clang 16.0.0 (clang-1600.0.26.3)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello")
Hello
>>> 1+4*2
9
>>> exit()
bash-3.2$
```

Pythonの実行

◎ スクリプト実行

- 適当なテキストエディタを開いて`print("Hello")`と入力
- `XXX.py`として保存（ここでは`sample1-1.py`）
- ターミナルで保存したディレクトリまで移動して`python3 sample1-1.py`を実行

sample1-1.pyの実行 → 
出力 →

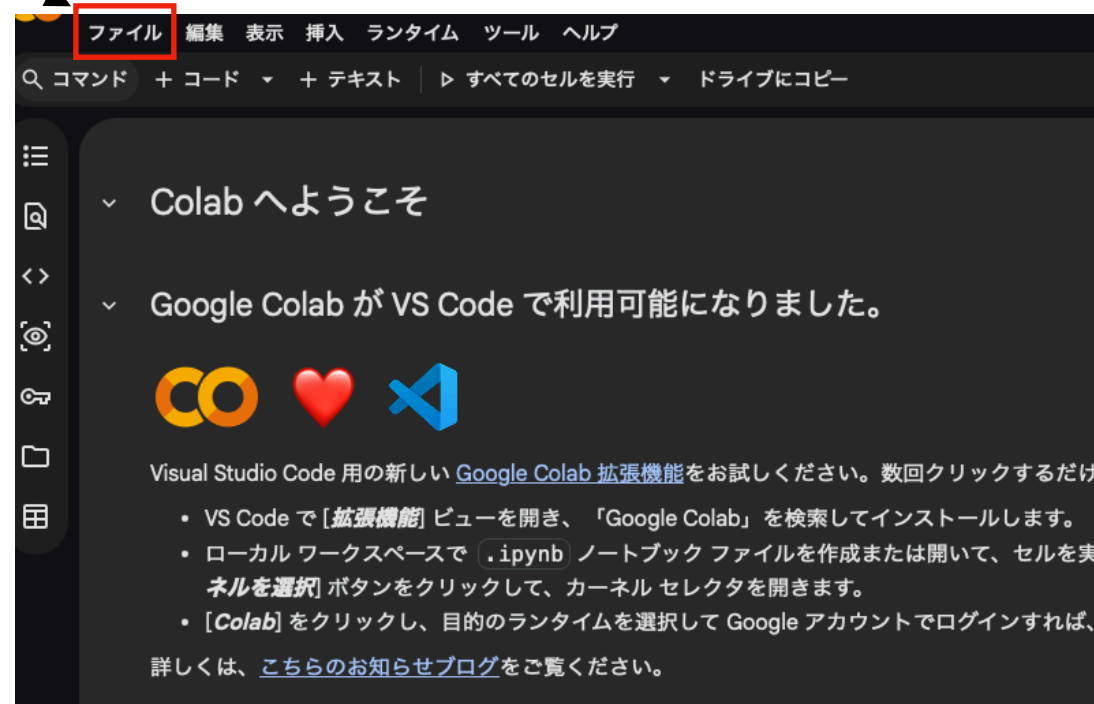
The image shows a terminal window with a black background and green text. The first line is the command prompt `bash-3.2$` followed by the command `python3 sample1-1.py`. The second line shows the output `Hello`. The third line shows the command prompt `bash-3.2$` followed by a green cursor block.

- スクリプト実行では`print`関数などで明示的に出力することを書かないとターミナル出力はされない
- 長いコードを実行したいときや他の人とコードを共有する時にはスクリプト実行の方が便利

Pythonの実行

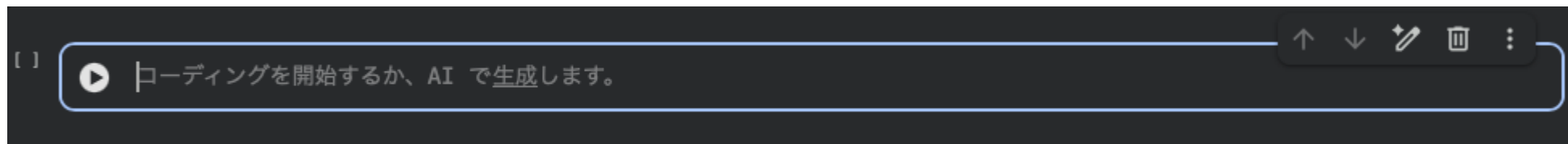
◎ Google Colabでの実行

- <https://colab.research.google.com/?hl=ja#scrollTo=1edGr7JPAZyr> へ移動
- google アカウントでログイン
- 「ファイル」から「ドライブの新しいノートブック」を選択

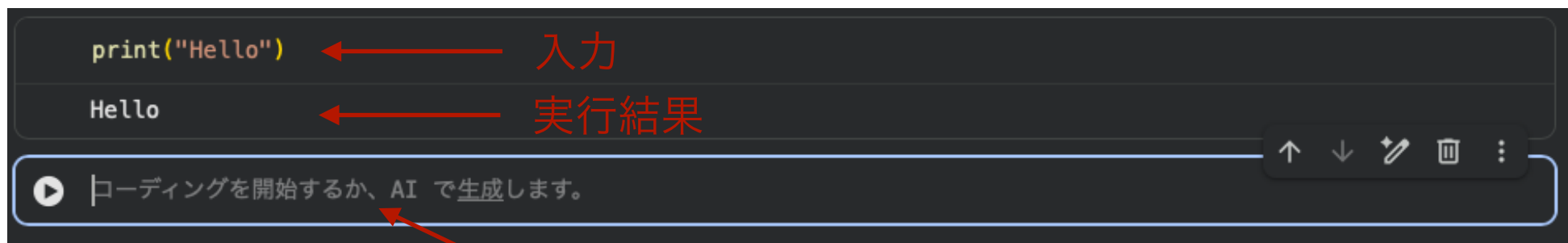


Pythonの実行

- 以下の枠が出てくる



- (今回は練習なのでAIを使わずに自分で打ち込んでみましょう)
- 枠内にprint("Hello")と入力。shiftキーを押しながらエンターキーを押すと実行される



次を入力できる

Pythonの基本文法

◎ 文の区切り

- Pythonは改行で文の区切り
- コードが長くなり改行でも文を区切りたくない時にはバックスラッシュ”\”、または括弧を使用

バックスラッシュを使わないで改行するとx=1+2+3として実行される

バックスラッシュつき或いは括弧で括って改行するとx=1+2+3+4+5+6として実行される

```
x=1+2+3
+4+5+6
print(x)
```

6

```
x=1+2+3\
+4+5+6
print(x)
```

21

```
x=(1+2+3
    +4+5+6)
print(x)
```

21

Pythonの基本文法

*インデントの意味はプログラミング言語によって大きく異なることに注意

◎ インデント（字下げ）

- Pythonではインデントでコードの構造を表す
- 特に制御構文ではインデントで構造を表す
- インデントの大きさは使用するエディタの設定などによるが同じコードでは同じ大きさを使うのが良い
- むやみやたらにインデントを使うのはNG

条件を満たした時の処理
と満たさなかった処理を
インデントを用いて字下
げする。



```
x=10
if x>5:
    print("x > 5 !!")
else:
    print("x <= 5 !!")

x > 5 !!
```

Pythonの基本文法

#以下はコメント

実行はされない

◎ コメント

- コメントはシャープ記号#で始める
- コメントは他の人や将来の自分がコードを読みやすくするために使います。ただし自明なコードに過剰なコメントは避けるべきです。（コードを編集していく中でとコードとコメントが乖離するなどして負の効果を与えることがある。）

```
x=10 #初期化
if x>5: # xが5より大きい
    print("x > 5 !!")
else:
    print("x <= 5 !!")

x > 5 !!
```

◎ 大文字と小文字の区別

- Pythonでは大文字と小文字が区別されます。

```
var=1
Var=2
print(var)
print(Var)

1
2
```

Pythonの基本文法

◎ 変数

- Pythonは動的型付けを採用しているため変数の宣言時に型を明示する必要がない。（一方C言語などは静的型付けで変数宣言時に型を明示する。）
- 変数名は英字、数字、アンダースコア(_)を使える。（数字で始めることはできない）
- いくつかの予約語(ifなど)は変数名として使えない。
- 宣言した変数はdelを用いて削除できる。
- Pythonでは変数を定数宣言する方法はない。（慣習的に、変数名を全て大文字で書くことで定数であることを示すことがある。）

delでxを削除した後に
xを表示しようとする
とエラーが出てくる

x,y,zは整数として扱われる

```
x=1
y=2
z = x+y
print(z)
x="Hello"
print(x)
```

3
Hello

代入時にxの型が整数から文字列に変わる

```
x=1
print(x)
del x
print(x)
```

1

NameError Traceback (most recent call last)
/tmp/ipython-input-3685188580.py in <cell line: 0>()
 2 print(x)
 3 del x
----> 4 print(x)

NameError: name 'x' is not defined

主な組み込み型

◎ 主な組み込み型

整数型	整数を表す。(2,-1,0など)
浮動小数点型	小数を表す。(3.14,-14.1,0.0,1.0など)
複素数型	複素数を表す。(1+2j,3j,-3jなど)
文字列型	文字の並びを表す。("Hello","", "(空文字列)など)
リスト型	複数の値を順序付けて格納する。([1,2,3],["a","b",4]など)
辞書型	キーと値のペアを格納する。("name":"Alice", "age":30)
タプル型	複数の値を順序付けて格納する(変更不可)。 ((1,2,3),("a","b",4)など)
論理(ブール)型	真偽値を表す。(True, False)

主な組み込み型

◎ 論理型

- True、Falseを表す
- None、0(数値)、空文字列、空リストなどはFalseと見なされる

空文字列

```
x=""  
if x:  
    print("x is not empty")  
else:  
    print("x is empty")  
  
x is empty
```

◎ 整数型

- 整数を表す。(10進数、2進数、8進数、16進数での表現が可能)

2進数は先頭に0bをつける

8進数は先頭に0oをつける

16進数は先頭に0xをつける

```
a=10 # 10進数  
b=0b110 # 2進数  
c=0o163 # 8進数  
d=0xB # 16進数  
print(a)  
print(b)  
print(c)  
print(d)
```

```
10  
6  
115  
11
```


主な組み込み型

◎ 浮動小数点型

- 実数を表す。

指数表記も可能

```
x=2.3
y=1.2e3
z=1.2e-3
print(x)
print(y)
print(z)
```

```
2.3
1200.0
0.0012
```

◎ 複素数型

- 複素数を表す。
- jで虚数単位を表す。
- 基本的な複素数の四則演算が可能

```
a=1.2+4.1j
b=-1.+0.2j
c=a+b
print(a)
```

```
(1.2+4.1j)
```

◎ 文字列型

- 文字の並びを表す。
- シングルクォート(')或いはダブルクォート("")で囲む

+演算子で文字列を結合できる。

```
gname="Taro"
sname='Sato'
name = gname+" "+sname
print(name)
```

```
Taro Sato
```


主な組み込み型

- エスケープシーケンスで特別な文字を表現できる

<code>\'</code>	シングルクォート
<code>\"</code>	ダブルクォート
<code>\n</code>	改行
<code>\t</code>	タブ

```
str1="\ 'test\' "  
str2="\ "test\" "  
str3="test1\ntest2"  
str4="test3\ttest4"  
print(str1)  
print(str2)  
print(str3)  
print(str4)
```

```
'test'  
"test"  
test1  
test2  
test3  test4
```

- フォーマット文字列
 - 文字列内に変数の値を埋め込める。
 - 文字列の前にfをつける。

name, ageが埋め込まれる

```
name="Taro"  
age=10  
message=f"I am {name}. I am {age} years old."  
print(message)
```

```
I am Taro. I am 10 years old.
```

主な組み込み型

◎ リスト型

- 複数の値を順序付きで格納する。
 - []で囲んで,で区切って表現する。
 - 各要素は0から始まる整数でアクセスできる。
 - リストの中にリストを入れることも可能。
 - 要素の追加：append
 - 要素の削除：remove
- 配列fruitsの各要素を表示
- リストの入れ子によって行列を表現

```
fruits = ["apple", "banana", "cherry"]  
print(fruits[0])  
print(fruits[1])  
print(fruits[2])
```

```
apple  
banana  
cherry
```

```
matrix=[[1,2,3],[4,5,6],[7,8,9]]  
print(matrix[0])  
print(matrix[0][1])
```

```
[1, 2, 3]  
2
```

要素の追加と削除

```
fruits = ["apple", "banana", "cherry"]  
fruits.append("date")  
print(fruits)  
fruits.remove("banana")  
print(fruits)
```

```
['apple', 'banana', 'cherry', 'date']  
['apple', 'cherry', 'date']
```

主な組み込み型

◎ 辞書型

- キーと値のペアを格納する。
- キーと値をコロンで区切って各ペアをカンマで区切って宣言する。
- [key]で値を得る。
- キーと値のペアの削除：del

辞書型を宣言
“age”を削除
削除したキーにアクセス
するとエラーが出る

```
person = {"name": "Alice", "age": 30}
person["city"] = "New_York"
print(person)
del person["age"]
print(person)
print(person["name"])
print(person["age"])

{'name': 'Alice', 'age': 30, 'city': 'New_York'}
{'name': 'Alice', 'city': 'New_York'}
Alice
-----
KeyError                                Traceback (most recent call last)
/tmp/ipython-input-474412672.py in <cell line: 0>()
      5 print(person)
      6 print(person["name"])
----> 7 print(person["age"])

KeyError: 'age'
```

主な組み込み型

◎ タプル型

- 複数の値を順序付きで格納する。要素の変更はできない。（イミュータブルという）
- 丸括弧（）で囲み、カンマで区切って宣言する。

タプル型を宣言
値を変更しようとする
とエラーになる

```
point = (10, 20, 30)
print(point[0])
point[0] = 15
```

```
10
```

```
-----
TypeError                                Traceback (most recent call last)
/tmp/ipython-input-4171188944.py in <cell line: 0>()
      1 point = (10, 20, 30)
      2 print(point[0])
----> 3 point[0] = 15

TypeError: 'tuple' object does not support item assignment
```

- pythonは動的型付け言語なのでコード中で変数を書き換えていく中で変数の型がわかりにくくなることもあるかもしれません。その時にはtypeという関数で型（クラス）を確認することができます。

aをリストとして宣言
aの型を表示
a[1]の型を表示

```
a=[1,2]
print(type(a))
print(type(a[1]))

<class 'list'>
<class 'int'>
```

演算子

◎ 代表的な演算子

- 算術演算子：+, -, *, /
- 比較演算子：==, !=, <, >, <=, >=
- 論理演算子：and, or, not
- 代入演算子：=, +=, -=, *=, /=
- (ビット演算子)

演算子

◎ 算術演算子

- 数値同士の計算を行う。

+	加算
-	減算
*	乗算
/	除算
%	剰余（余り）
**	冪乗
//	商の整数分

```
a = 10
b = 3
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a % b)
print(a ** b)
print(a // b)
```

13
7
30
3.3333333333333335
1
1000
3

- 数値型以外でも算術演算を行える。その際の動作はデータ型によって異なる。(算術演算が定義されていないデータ型に対して算術演算をするとエラーとなる。)

```
str1= "Hello,_"
str2= "World!"
result = str1 + str2
print(result)
```

Hello,_World!

+演算子は文字列型を連結する。

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
result = list1 + list2
print(result)
```

[1, 2, 3, 4, 5, 6]

+演算子はリストを連結する。

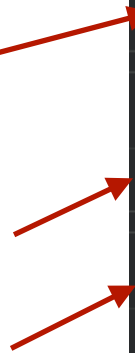
演算子

- 浮動小数点型の計算の注意
 - 打ち切り誤差：（無限小数などで）計算処理を途中で打ち切りことによる誤差
 - 桁落ち：値がほぼ等しい数値同士の引き算で発生する誤差
 - 情報落ち：大きい値と小さい値の足し算引き算で発生する誤差
 - これらの誤差をなくす（小さく）するライブラリもあるが計算の工夫で誤差が出てこないようにすることも大切。

```
print(0.2/3.0)
0.06666666666666667

print(1.000000000001-1.000000000000)
1.0000000082740371e-11

print(1e10-1e-10)
10000000000.0
```



演算子

◎ 比較演算子

- 数値同士の計算を行う。
- 条件分岐やループなどで使う。
- 整数や浮動小数同士以外でも使える。ただし挙動はデータ型によって異なるので正しく使う必要がある。

文字列は辞書順の比較

```
print(15>131)
print("15">"131")
print("ab">"aa")
```

```
False
True
True
```

数字を文字列として比較するときと数字として比較するときで挙動が異なる！！

==	等しい
!=	等しくない
<	小さい
>	大きい
<=	以下
>=	以上
is	同一性比較
is not	非同一性比較
in	含まれる
not in	含まれない

```
list1=[1,2,4]
list2=[1,2,3]
list3=[1,2]
print(list1<list2)
print(list1<list3)
```

```
False
False
```

リスト同士の比較では先頭から順番に要素比較を行い大小を決定。それで大小が決定しなければリストの長さで大小を決定

演算子

- 同一性比較と同値性比較を区別する。
 - 同一性比較：参照している実体そのものが等しいかを比較 (is演算子)
 - 同値性比較：参照している実体を持つ値が等しいかを比較 (==演算子)
- リスト型、辞書型などはミュータブル型（変更可能）と呼ばれる

list1とlist2の値が等しい

list1とlist2の実体は異なる

list1とlist2の実
体が同じになる

```
list1=[1,2,3]
list2=[1,2,3]
print(list1==list2)
print(list1 is list2)
print(id(list1))
print(id(list2))
list1=list2
print(list1 is list2)
print(id(list1))
```

True
False
136055033662912
136055033672448
True
136055033672448

id関数でそれぞれの変数の格
納場所の識別子が得られる

```
list1=[1,2,3]
list2=list1
list2.append(4)
print(list1)
```

[1, 2, 3, 4]

list1とlist2が同一の時list2を
更新するとlist1も更新される

演算子

- 数値型や文字列型はイミュータブル（変更不可）と呼ばれる。

イミュータブルな変数は別々に作成された
同じ値の変数が同一になる（ことがある）



```
a="data"  
b="data"  
print(a is b)
```

```
True
```

- イミュータブルな型でも同値であり同一でない場合もあり得る。

演算子

◎ 代入演算子

=	値を代入
+=	左辺+右辺を左辺に代入
-=	左辺-右辺を左辺に代入
*=	左辺*右辺を左辺に代入
/=	左辺/右辺を左辺に代入
//=	左辺/右辺の整数部を左辺に代入
%=	左辺/右辺の余りを左辺に代入
**=	左辺の右辺による冪乗を左辺に代入
&=	左辺と右辺の論理積を左辺に代入
^=	左辺と右辺の排他的論理和を左辺に代入
=	左辺と右辺の論理和を左辺に代入
>>=	左辺を右辺値だけ右シフトしたものを左辺に代入
<<=	左辺を右辺値だけ左シフトしたものを左辺に代入

演算子

- “=”演算子は参照渡し（=で変数を代入すると）

ミュータブルな型の場合

x,yは同じidを持つ
（実体が同じ）

yの値を変更するとy
の参照先が変わる。

```
x=1
y=x
print(id(x))
print(id(y))
y=2
print(id(x))
print(id(y))
```

```
11654376
11654376
11654376
11654408
```

data1,data2は同じid
を持つ（実体が同じ）

data2の値を変更して
もdata2の参照先が変
わらない。そのため
data1の値も変化する

イミュータブルな型の場合

```
data1=[1,2,3]
data2=data1
print(id(data1))
print(id(data2))
data2.append(4)
print(id(data1))
print(id(data2))
print(data1)
```

```
136134948899712
136134948899712
136134948899712
136134948899712
[1, 2, 3, 4]
```

同じ値には同じidが
割り当てられる

```
z=1
w=1
print(id(z))
print(id(w))
```

```
11654376
11654376
```

同じ値でも異なるid
が割り当てられる。

```
data3=[1,2,3]
data4=[1,2,3]
print(id(data3))
print(id(data4))
```

```
136134944934656
136135340262080
```

演算子

- アンバック代入

```
data=[1,2,3,4,5]  
a,b,c,d,e=data  
print(b)
```

```
2
```

左辺に右辺のリストの要素が順番に代入される

左辺の変数の数が右辺のリストの要素数と異なる場合にはエラーが出る

```
a,b=data
```

```
-----  
ValueError                                Traceback (most recent call last)  
/tmp/ipython-input-140862892.py in <cell line: 0>()  
----> 1 a,b=data  
  
ValueError: too many values to unpack (expected 2)
```

```
data=[1,2,3,4,5]  
a,b,*c = data  
print(c)
```

```
[3, 4, 5]
```

*cで残りの変数を一括してリストとして受け取る

演算子

● 論理演算子

and	論理積
or	論理和
^	排他的論理和
not	否定

		and	or	^
T	T	T	T	F
T	F	F	T	T
F	T	F	T	T
F	F	F	F	F

● ビット演算子

&	論理積
	論理和
^	排他的論理和
~	否定
<<	ビットを左シフト
>>	ビットを右シフト

```
x=0b111
print(x)
print(x<<1)
print(x>>1)
```

7
14
3

$(111)_2 \rightarrow (1110)_2$

$(111)_2 \rightarrow (11)_2$